

PROBABILITY

for Machine Learning, Deep Learning, AI & Data Science

Beginner-Friendly • Intuition-Driven • 65+ Pages

From basic probability rules to Bayes' Theorem, distributions,
MLE, entropy, and real ML applications

Python examples with NumPy, SciPy, Matplotlib & Seaborn

Author: Your Name | Date: 2025 | Version 1.0

Designed for ML students, data scientists, and AI practitioners

Table of Contents

Introduction to Probability for ML & AI	5
What is Probability?.....	5
Why Probability is Critical in Machine Learning	5
Probability vs. Statistics vs. Machine Learning	6
Chapter 1: Fundamentals of Probability	7
1.1 Random Experiments, Sample Spaces & Events	7
Sample Space (Ω)	7
Events	7
1.2 The Basic Probability Formula.....	7
Step-by-Step Example	8
Chapter 2: The Rules of Probability	10
2.1 The Complement Rule.....	10
2.2 The Addition Rule.....	10
Step-by-Step Example: Cards.....	10
2.3 The Multiplication Rule	11
Chapter 3: Types of Events.....	13
3.1 Independent Events.....	13
3.2 Dependent Events.....	13
Example: Drawing Without Replacement.....	13
3.3 Mutually Exclusive Events	13
Chapter 4: Conditional Probability — The Heart of ML	15
4.1 What is Conditional Probability?	15
Intuitive Explanation.....	15
Step-by-Step Example	15
4.2 The Chain Rule of Probability	16
Chapter 5: Bayes' Theorem — The Core of Probabilistic ML	17
5.1 The Intuition: Updating Beliefs.....	17
5.2 Solving the Medical Test Problem	17
5.3 Applications of Bayes' Theorem in ML	18
5.4 Law of Total Probability	18
Chapter 6: Random Variables.....	20
6.1 What is a Random Variable?	20
6.2 Discrete vs. Continuous Random Variables.....	20
Chapter 7: Expectation, Variance & Standard Deviation	22
7.1 Expected Value (Mean)	22

Example: Fair Die	22
ML Interpretation.....	22
7.2 Variance and Standard Deviation	22
Chapter 8: Discrete Probability Distributions	24
8.1 Bernoulli Distribution.....	24
8.2 Binomial Distribution.....	24
Example: Email Campaign.....	24
8.3 Poisson Distribution.....	25
Chapter 9: Continuous Probability Distributions	26
9.1 Uniform Distribution.....	26
9.2 Normal (Gaussian) Distribution — The Most Important Distribution in ML	26
The Standard Normal Distribution $N(0, 1)$	26
The 68-95-99.7 Rule	27
Why the Normal Distribution is Central to ML.....	27
Chapter 10: PMF, PDF & CDF — Describing Distributions	28
10.1 Probability Mass Function (PMF).....	28
10.2 Probability Density Function (PDF).....	28
10.3 Cumulative Distribution Function (CDF).....	28
Chapter 11: Joint, Marginal & Conditional Distributions.....	30
11.1 Joint Probability Distribution	30
Example: Weather & Traffic	30
11.2 Marginal Distribution.....	30
11.3 Independence of Random Variables.....	30
Chapter 12: Likelihood & Maximum Likelihood Estimation	32
12.1 Probability vs. Likelihood	32
12.2 Maximum Likelihood Estimation (MLE).....	32
MLE for a Coin: Step-by-Step	33
12.3 MLE and Neural Network Training.....	34
Chapter 13: Entropy & Information Theory.....	35
13.1 What is Entropy?	35
Entropy Examples.....	35
13.2 Cross-Entropy	35
13.3 KL Divergence.....	36
13.4 Information Gain — Decision Trees.....	36
Chapter 14: Law of Large Numbers & Central Limit Theorem	38
14.1 The Law of Large Numbers (LLN)	38

14.2 Central Limit Theorem (CLT)	38
Chapter 15: Naive Bayes Classifier	41
15.1 What is Naive Bayes?	41
15.2 Step-by-Step: Spam Detection	41
Chapter 16: Gaussian (Normal) Models in Machine Learning	43
16.1 Gaussian Naive Bayes	43
16.2 Gaussian Mixture Models (GMM)	43
16.3 Normal Distribution in Neural Network Weight Initialisation.....	44
Chapter 17: Probabilistic Thinking in AI.....	45
17.1 Uncertainty in AI Systems.....	45
17.2 Probabilistic ML Models vs. Deterministic	45
17.3 Softmax — Converting Scores to Probabilities	46
17.4 Bayesian Deep Learning	46
Chapter 18: Probability with Python — Practical Applications	47
18.1 Simulating Basic Probability Experiments	47
18.2 Working with Distributions	47
18.3 Visualising Bayes' Theorem Updates	48
Chapter 19: Mini Project — Build a Naive Bayes Text Classifier	50
19.1 Project Overview	50
19.2 Step-by-Step Implementation	50
19.3 Using Scikit-learn Pipeline	51
Chapter 20: Summary & Probability Cheat Sheet	53
20.1 Complete Formula Reference.....	53
20.2 Distributions Quick Reference	53
20.3 Learning Roadmap — What to Study Next	54
Appendix: Key Python Libraries for Probability	55

Introduction to Probability for ML & AI

What is Probability?

Probability is the mathematical study of uncertainty. It gives us a precise language for describing how likely events are to occur. In everyday life, we use probability constantly without realising it: 'There's a 70% chance of rain today,' or 'I'll probably finish this task by Friday.' Probability formalises these intuitions into numbers between 0 and 1.

At its core, a probability of 0 means something is impossible — it will never happen. A probability of 1 means something is certain — it will always happen. Everything else falls in between. A fair coin has a 0.5 probability of landing heads; a six-sided die has a $1/6 \approx 0.167$ probability of landing on any given number.

☐ **Probability = Measuring Uncertainty**

Probability answers one fundamental question: 'How confident should I be that this will happen?' It is a number between 0 (impossible) and 1 (certain).
0.5 = coin flip — completely uncertain. 0.99 = very likely but not guaranteed.
Every machine learning model is, at its heart, a probability-generating machine.

Why Probability is Critical in Machine Learning

Machine learning models don't output simple yes/no answers — they output probabilities. A spam classifier doesn't say 'this is spam'; it says 'this has a 97.3% chance of being spam.' A medical image model doesn't say 'the patient has cancer'; it says 'the probability of malignancy is 0.82.' This distinction is crucial in practice.

Without probability theory, we cannot build, train, evaluate, or deploy machine learning systems effectively. Here is how probability permeates every layer of ML and AI:

Probability Concept	ML/AI Application	Deep Learning Use	Data Science Use
Bayes' Theorem	Naive Bayes classifier	Bayesian neural nets	Prior belief updating
Conditional Probability	Classification models	Attention mechanisms	Conditional statistics
Distributions	Generative models (GANs, VAEs)	Weight initialisation	Data modelling
MLE	Logistic regression	Loss function derivation	Parameter estimation
Entropy	Decision tree splits	Cross-entropy loss	Feature information
Law of Large Numbers	Training convergence	Batch statistics	Sampling reliability
Central Limit Theorem	Confidence intervals	Gradient noise analysis	A/B testing

Joint Probability	Latent variable models	Variational autoencoders	Multivariate analysis
-------------------	------------------------	--------------------------	-----------------------

Probability vs. Statistics vs. Machine Learning

These three fields are deeply related. Probability is the theoretical foundation that assumes you know the model and asks: 'What data is likely?' Statistics works in reverse — given data, it asks: 'What model likely generated this?' Machine learning takes both and asks: 'How can we build a system that improves at a task using data?'

□ Real-World Analogy

Imagine a bag with 3 red and 2 blue marbles.

PROBABILITY asks: 'If I draw one marble, what's the chance it's red?' (Answer: $3/5 = 0.6$)

STATISTICS asks: 'I drew 10 marbles and got 6 red. How many red marbles are in the bag?'

MACHINE LEARNING asks: 'Train a model so it predicts the colour before I draw.'

Chapter 1: Fundamentals of Probability

1.1 Random Experiments, Sample Spaces & Events

A random experiment is any process whose outcome cannot be predicted with certainty before it occurs, even if we know all the rules. The classic examples are coin flips, dice rolls, and card draws — but in ML, random experiments include sampling a training data point, initialising neural network weights, or choosing a random batch.

Sample Space (Ω)

The sample space is the set of ALL possible outcomes of a random experiment. It is typically denoted by the Greek letter Ω (omega).

Sample Space Definition:
 $\Omega = \{ \text{all possible outcomes of the experiment} \}$

Experiment	Sample Space Ω
Flip a fair coin	$\Omega = \{ \text{Heads, Tails} \}$
Roll a six-sided die	$\Omega = \{ 1, 2, 3, 4, 5, 6 \}$
Draw a card from standard deck	$\Omega = \{ \text{all 52 cards} \}$
Measure temperature tomorrow	$\Omega = \{ \text{all real numbers, e.g. } -50^{\circ}\text{C to } 60^{\circ}\text{C} \}$
Predict if email is spam	$\Omega = \{ \text{spam, not spam} \}$
A model predicts house price	$\Omega = \{ \text{all positive real numbers} \}$

Events

An event is a subset of the sample space — it is a collection of outcomes we are interested in. Events are usually denoted by capital letters A, B, C, etc.

□

Events — Key Examples

Experiment: Roll a die. Sample Space: $\Omega = \{1, 2, 3, 4, 5, 6\}$

Event A = 'Roll an even number' $\rightarrow A = \{2, 4, 6\}$

Event B = 'Roll a number greater than 4' $\rightarrow B = \{5, 6\}$

Event C = 'Roll a 1' $\rightarrow C = \{1\}$ (a simple event with one outcome)

Event D = 'Roll a 7' $\rightarrow D = \{\} = \emptyset$ (impossible event — empty set)

1.2 The Basic Probability Formula

For experiments where all outcomes are equally likely (called a uniform probability space), the probability of an event A is:

Basic Probability Formula:

$$P(A) = (\text{Number of favourable outcomes}) / (\text{Total number of outcomes in } \Omega)$$

This formula only works when each outcome is equally likely. It fails for unfair coins, biased dice, or any real-world scenario where outcomes have different probabilities.

Step-by-Step Example

1. Experiment: Roll a fair six-sided die.
2. Sample space: $\Omega = \{1, 2, 3, 4, 5, 6\}$, total outcomes = 6
3. Event A = 'Roll an even number' = $\{2, 4, 6\}$, favourable outcomes = 3
4. $P(A) = 3/6 = 0.5 \rightarrow 50\%$ chance of rolling even

```
import numpy as np
import matplotlib.pyplot as plt

# Simulate 10,000 dice rolls
np.random.seed(42)
rolls = np.random.randint(1, 7, size=10000)

# Probability of rolling an even number
even_rolls = rolls[rolls % 2 == 0]
prob_even = len(even_rolls) / len(rolls)
print(f'Simulated P(even) = {prob_even:.4f}') # Should be ~0.5

# Plot distribution of rolls
plt.figure(figsize=(8, 4))
plt.hist(rolls, bins=range(1, 8), align='left', rwidth=0.8, color='steelblue',
         edgecolor='white')
plt.xlabel('Dice Value')
plt.ylabel('Frequency')
plt.title('Distribution of 10,000 Dice Rolls')
plt.xticks(range(1, 7))
plt.show()
```

□ Mini Exercises

1. A bag contains 4 red, 3 blue, and 2 green marbles. What is the probability of drawing a blue marble?
2. In a class of 30 students, 12 are left-handed. What is $P(\text{left-handed})$?
3. A fair coin is flipped twice. What is the sample space? What is $P(\text{at least one head})$?

✓ **Answers:**

1. $P(\text{blue}) = 3/(4+3+2) = 3/9 = 1/3 \approx 0.333$
2. $P(\text{left-handed}) = 12/30 = 0.4 = 40\%$
3. $\Omega = \{HH, HT, TH, TT\}$. $P(\text{at least one head}) = 3/4 = 0.75$

Chapter 2: The Rules of Probability

2.1 The Complement Rule

The complement of event A (written A' or A^c or 'not A') is the set of all outcomes NOT in A. If you know $P(A)$, you immediately know the probability that A does NOT happen.

Complement Rule:

$$P(A') = 1 - P(A)$$

This is extremely useful: if finding $P(A)$ directly is hard, sometimes it is easier to find $P(A')$ and subtract from 1.

□ Analogy — Weather Forecast

The weather app says there's a 30% chance of rain.

$$\rightarrow P(\text{rain}) = 0.30$$

$$\rightarrow P(\text{no rain}) = 1 - 0.30 = 0.70$$

In ML: If a classifier says $P(\text{spam}) = 0.85$, then $P(\text{not spam}) = 1 - 0.85 = 0.15$

2.2 The Addition Rule

When you want the probability that event A OR event B happens, you use the addition rule. There are two versions depending on whether the events overlap.

Addition Rule (General):

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

We subtract $P(A \cap B)$ because outcomes in BOTH A and B would be counted twice without the correction.

Addition Rule (Mutually Exclusive Events):

$$P(A \cup B) = P(A) + P(B) \quad [\text{only when } A \cap B = \emptyset]$$

Mutually exclusive events cannot both occur at the same time. Rolling a 2 and rolling a 5 on one die roll are mutually exclusive.

Step-by-Step Example: Cards

From a standard 52-card deck, what is the probability of drawing a King OR a Heart?

5. $P(\text{King}) = 4/52$ (there are 4 kings)
6. $P(\text{Heart}) = 13/52$ (there are 13 hearts)
7. $P(\text{King AND Heart}) = P(\text{King of Hearts}) = 1/52$
8. $P(\text{King OR Heart}) = 4/52 + 13/52 - 1/52 = 16/52 \approx 0.308$

2.3 The Multiplication Rule

When you want the probability that BOTH event A AND event B occur, you use the multiplication rule. Again there are two versions.

Multiplication Rule (Independent Events):

$$P(A \cap B) = P(A) \times P(B)$$

Multiplication Rule (Dependent Events):

$$P(A \cap B) = P(A) \times P(B | A)$$

$P(B | A)$ is read as 'the probability of B given A has already occurred.' This is conditional probability, which we will explore deeply in Chapter 4.

□ Summary of the Three Core Rules

COMPLEMENT: $P(\text{not } A) = 1 - P(A)$

ADDITION: $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$

MULTIPLICATION (independent): $P(A \text{ and } B) = P(A) \times P(B)$

These three rules, combined with Bayes' Theorem, power most probabilistic ML systems.

```
# Demonstrating probability rules in Python
import numpy as np

np.random.seed(0)
n = 100000

# Simulate drawing cards (simplify: 1=Ace...13=King, suits 1-4)
ranks = np.random.randint(1, 14, n) # rank 1-13
suits = np.random.randint(1, 5, n)  # suit 1-4 (4=Hearts)

king   = (ranks == 13)
heart  = (suits == 4)
king_or_heart = king | heart
```

```
print(f'P(King)           = {king.mean():.4f}')    # ~0.0769
print(f'P(Heart)          = {heart.mean():.4f}')    # ~0.2500
print(f'P(King|Heart)     = {(king & heart).mean():.4f}') # ~0.0192
print(f'P(King OR Heart)= {king_or_heart.mean():.4f}') # ~0.3077
```

Chapter 3: Types of Events

3.1 Independent Events

Two events A and B are independent if the occurrence of one does NOT affect the probability of the other. The mathematical definition is:

Independence Definition:

$$P(A \cap B) = P(A) \times P(B) \quad \Leftrightarrow \quad A \text{ and } B \text{ are independent}$$

Equivalently, A and B are independent if $P(A | B) = P(A)$ — knowing B happened gives no information about A.

□ Real-World Examples of Independence

- ✓INDEPENDENT: Flipping a coin AND rolling a die — one doesn't affect the other.
- ✓INDEPENDENT: Whether it rains in Tokyo AND whether it snows in London on the same day.
- ✗NOT INDEPENDENT: Drawing two cards WITHOUT replacement — the second draw depends on the first.
- ✗NOT INDEPENDENT: Email word count AND probability of being spam — they are related.

Independence is a critical assumption in many ML algorithms. Naive Bayes, for instance, assumes all features are independent given the class label. This assumption is almost never perfectly true in real data, but the classifier surprisingly still works well.

3.2 Dependent Events

Events A and B are dependent when the outcome of one affects the probability of the other. Most real-world events are actually dependent — this is why we need conditional probability.

Example: Drawing Without Replacement

A bag contains 5 red balls and 3 blue balls. You draw two balls without replacement.

9. $P(\text{first ball is red}) = 5/8$
10. Given the first ball was red, $P(\text{second ball is red}) = 4/7$ (only 4 red left, 7 total left)
11. $P(\text{both red}) = P(1\text{st red}) \times P(2\text{nd red} | 1\text{st red}) = 5/8 \times 4/7 = 20/56 \approx 0.357$

3.3 Mutually Exclusive Events

Two events are mutually exclusive (also called disjoint) if they CANNOT both occur at the same time. If A happens, B cannot, and vice versa.

Mutually Exclusive:

$$P(A \cap B) = 0 \quad \text{and} \quad P(A \cup B) = P(A) + P(B)$$

Mutually Exclusive ✔	NOT Mutually Exclusive ✕
Rolling a 3 and a 5 on ONE die roll	Rolling an even number and rolling a 2
Classifying an email as spam AND not spam	A user being male AND being over 25
A coin landing heads AND tails simultaneously	It raining AND it being cloudy
A number being both <3 and >5	A student passing AND being tall

☐ **Common Confusion: Independent ≠ Mutually Exclusive**

Students often confuse these two concepts. They are VERY different:

MUTUALLY EXCLUSIVE: $P(A \cap B) = 0$ — they cannot both happen.

INDEPENDENT: $P(A \cap B) = P(A) \times P(B)$ — they can both happen, but don't influence each other.

In fact, if A and B are mutually exclusive AND both have $P > 0$, they are DEPENDENT (because if A happens, you know B did NOT happen — so A gives info about B).

Chapter 4: Conditional Probability — The Heart of ML

4.1 What is Conditional Probability?

Conditional probability asks: 'Given that we already know event B occurred, what is the probability that event A also occurred?' It updates our beliefs based on new information. This is the foundation of Bayesian reasoning and most probabilistic ML systems.

Imagine you are trying to predict whether a patient has flu. If you know nothing, you use the general population rate. But if you learn the patient has a fever, you update your estimate upward significantly. That update is conditional probability.

Conditional Probability Formula:

$$P(A \mid B) = P(A \cap B) / P(B) \quad [\text{provided } P(B) > 0]$$

Read $P(A \mid B)$ as 'the probability of A given B.' The vertical bar $|$ always means 'given' or 'assuming.'

Intuitive Explanation

Think of conditional probability as zooming in. When we condition on B, we restrict our universe to only the outcomes where B has already occurred. Within that restricted universe, we then ask: 'What fraction of these outcomes are also in A?'

Step-by-Step Example

A group of 100 students: 60 study Statistics, 40 study Programming, and 25 study BOTH.

12. $P(\text{Statistics}) = 60/100 = 0.6$

13. $P(\text{Programming}) = 40/100 = 0.4$

14. $P(\text{Both}) = 25/100 = 0.25$

15. $P(\text{Programming} \mid \text{Statistics}) = P(\text{Both}) / P(\text{Statistics}) = 0.25 / 0.6 = 0.417$

→ Knowing a student studies Statistics raises our estimate that they also study Programming from 40% to 41.7%.

```
# Conditional Probability Example
import numpy as np

np.random.seed(42)
n = 100000

# Simulate student choices (correlated)
stats = np.random.random(n) < 0.60          # 60% study Statistics
prog = np.where(stats,
                  np.random.random(n) < 0.417, # 41.7% given Stats
```

```
np.random.random(n) < 0.25)    # 25% without Stats

both = stats & prog

p_stats = stats.mean()
p_prog = prog.mean()
p_both = both.mean()
p_prog_given_stats = both.sum() / stats.sum()

print(f'P(Stats)           = {p_stats:.3f}')
print(f'P(Prog)            = {p_prog:.3f}')
print(f'P(Both)            = {p_both:.3f}')
print(f'P(Prog | Stats)    = {p_prog_given_stats:.3f}') # ~0.417
```

4.2 The Chain Rule of Probability

By rearranging the conditional probability formula, we get the multiplication rule in its general form:

Chain Rule:

$$P(A \cap B) = P(A | B) \times P(B) = P(B | A) \times P(A)$$

For multiple events, this extends naturally:

Chain Rule for Multiple Events:

$$P(A \cap B \cap C) = P(A) \times P(B | A) \times P(C | A \cap B)$$

□ Mini Exercises

1. A die is rolled. Given that the result is odd, what is the probability it is a 3?
2. In a dataset, 40% of users clicked an ad. 30% of ad-clickers made a purchase. What is $P(\text{click AND purchase})$?
3. $P(A) = 0.5$, $P(B|A) = 0.6$. What is $P(A \cap B)$?

✓ Answers:

1. Odd numbers are $\{1, 3, 5\}$. Given odd, $P(3|\text{odd}) = 1/3 \approx 0.333$
2. $P(\text{purchase} \cap \text{click}) = P(\text{click}) \times P(\text{purchase}|\text{click}) = 0.40 \times 0.30 = 0.12$
3. $P(A \cap B) = P(A) \times P(B|A) = 0.5 \times 0.6 = 0.30$

Chapter 5: Bayes' Theorem — The Core of Probabilistic ML

5.1 The Intuition: Updating Beliefs

Bayes' Theorem tells us how to update our beliefs in light of new evidence. Before we see any data, we have a 'prior belief' about something. After we observe evidence, we update that belief to a 'posterior belief.' This update process is the foundation of Bayesian machine learning.

❑

The Medical Test Analogy

Problem: A disease affects 1% of the population. A test is 99% accurate (99% true positive, 1% false positive).

You test positive. What is the probability you actually have the disease?

Intuition says: 99% accurate test → 99% chance you're sick. But this intuition is **WRONG**.

Bayes' Theorem shows the actual answer is only about 50%! (Because the disease is rare.)

This is why Bayes is so important — human intuition fails without it.

Bayes' Theorem:

$$P(A \mid B) = [P(B \mid A) \times P(A)] / P(B)$$

Term	Meaning in Bayes' Theorem
$P(A \mid B)$ — Posterior	Updated probability of A after seeing evidence B. What we want to find.
$P(B \mid A)$ — Likelihood	How likely is evidence B if A were true? The model's fit.
$P(A)$ — Prior	Our initial belief about A before seeing any evidence.
$P(B)$ — Marginal Likelihood	Overall probability of seeing evidence B (normalising constant).

5.2 Solving the Medical Test Problem

Let A = 'Has disease', B = 'Tests positive'.

16. $P(A) = 0.01$ (1% of population has disease — the prior)
17. $P(B \mid A) = 0.99$ (test is positive 99% of time if you have disease — the likelihood)
18. $P(B \mid \text{not } A) = 0.01$ (1% false positive rate)
19. $P(B) = P(B|A) \times P(A) + P(B|\text{not } A) \times P(\text{not } A)$
 $= 0.99 \times 0.01 + 0.01 \times 0.99 = 0.0099 + 0.0099 = 0.0198$
20. $P(A \mid B) = (0.99 \times 0.01) / 0.0198 = 0.0099 / 0.0198 = 0.50$

Even with a 99% accurate test, if you test positive for a rare disease (1% prevalence), there is only a 50% chance you actually have it. This is why Bayes is so powerful — it prevents us from over-updating based on evidence without considering the prior.

```
# Bayes' Theorem - Medical Test Example

# Known values
p_disease      = 0.01    # Prior: P(A)
p_pos_given_d  = 0.99    # Likelihood: P(B|A)
p_pos_given_no_d = 0.01  # False positive: P(B|not A)
p_no_disease   = 1 - p_disease # P(not A)

# Marginal probability of testing positive: P(B)
p_positive = p_pos_given_d * p_disease + p_pos_given_no_d * p_no_disease

# Bayes' Theorem: P(A|B)
p_disease_given_pos = (p_pos_given_d * p_disease) / p_positive

print(f'P(disease | positive test) = {p_disease_given_pos:.4f}')
print(f'That is only {p_disease_given_pos*100:.1f}%!')
print(f'P(B) = {p_positive:.4f}')
```

5.3 Applications of Bayes' Theorem in ML

ML Application	How Bayes is Applied
Spam Filtering	$P(\text{spam} \mid \text{words in email}) \propto P(\text{words} \mid \text{spam}) \times P(\text{spam})$
Medical Diagnosis AI	$P(\text{disease} \mid \text{symptoms}) \propto P(\text{symptoms} \mid \text{disease}) \times P(\text{disease})$
Document Classification	$P(\text{category} \mid \text{document}) \propto P(\text{document} \mid \text{category}) \times P(\text{category})$
Fraud Detection	$P(\text{fraud} \mid \text{transaction}) \propto P(\text{transaction} \mid \text{fraud}) \times P(\text{fraud})$
Robot Localisation	$P(\text{location} \mid \text{sensor data}) \propto P(\text{sensor} \mid \text{location}) \times P(\text{location})$
Bayesian Neural Networks	$P(\text{weights} \mid \text{data}) \propto P(\text{data} \mid \text{weights}) \times P(\text{weights})$

5.4 Law of Total Probability

Bayes' Theorem requires $P(B)$, the probability of the evidence. When B can occur across multiple scenarios, we use the Law of Total Probability to compute it:

Law of Total Probability:

$P(B) = \sum P(B | A_i) \times P(A_i)$ for all mutually exclusive and exhaustive events A_i

In the medical example, $P(\text{positive test}) = P(\text{positive} | \text{disease}) \times P(\text{disease}) + P(\text{positive} | \text{no disease}) \times P(\text{no disease})$.

Chapter 6: Random Variables

6.1 What is a Random Variable?

A random variable is a function that assigns a numerical value to each outcome in the sample space. It is our way of turning qualitative random outcomes into numbers that we can calculate with.

Think of it this way: instead of saying 'heads' or 'tails,' we define $X = 1$ for heads and $X = 0$ for tails. Now we can calculate averages, variances, and so on.

□ Random Variable — Definition

A random variable X is a mapping from the sample space Ω to real numbers $\mathbb{R}$. It is 'random' because before the experiment, we don't know which value it will take. After the experiment, X takes a specific value — called a realisation. Random variables are typically denoted by uppercase letters: X , Y , Z . Their realisations (specific values) are denoted by lowercase: x , y , z .

6.2 Discrete vs. Continuous Random Variables

This is one of the most important distinctions in probability theory. The type of random variable determines which mathematical tools we use.

Discrete Random Variable	Continuous Random Variable
Takes a countable number of values	Takes any value in a range (uncountably infinite)
Examples: number of heads in 10 flips, die roll result, number of emails per day	Examples: height, weight, temperature, time, probability score
$P(X = x)$ is well-defined and positive	$P(X = \text{exact value}) = 0$; we use intervals instead
Described by Probability Mass Function (PMF)	Described by Probability Density Function (PDF)
Sum of all probabilities = 1	Integral of PDF over all values = 1

```
import numpy as np
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Discrete: Die roll
values = [1, 2, 3, 4, 5, 6]
probs = [1/6] * 6
ax1.bar(values, probs, color='steelblue', edgecolor='white', width=0.6)
ax1.set_title('Discrete: Fair Die PMF')
```

```
ax1.set_xlabel('Outcome') ; ax1.set_ylabel('P(X = x)')

# Continuous: Normal distribution
from scipy.stats import norm
x = np.linspace(-4, 4, 300)
ax2.plot(x, norm.pdf(x), color='crimson', lw=2)
ax2.fill_between(x, norm.pdf(x), alpha=0.3, color='crimson')
ax2.set_title('Continuous: Normal Distribution PDF')
ax2.set_xlabel('x') ; ax2.set_ylabel('f(x)')
plt.tight_layout() ; plt.show()
```

Chapter 7: Expectation, Variance & Standard Deviation

7.1 Expected Value (Mean)

The expected value (or expectation) $E[X]$ is the average value that a random variable takes over many repetitions of the experiment. It is the centre of mass of the probability distribution — where the distribution balances.

Expected Value (Discrete):

$$E[X] = \sum x \cdot P(X = x) \quad \text{for all values } x$$

Expected Value (Continuous):

$$E[X] = \int x \cdot f(x) \, dx \quad \text{over all } x$$

Example: Fair Die

$$E[\text{die roll}] = 1 \times (1/6) + 2 \times (1/6) + 3 \times (1/6) + 4 \times (1/6) + 5 \times (1/6) + 6 \times (1/6) = 21/6 = 3.5$$

Note: $E[X] = 3.5$, but you can never actually roll a 3.5 on a fair die. The expected value is an average over many trials, not a single likely outcome.

ML Interpretation

In machine learning, the expected value has several critical roles: the mean of a feature distribution, the expected loss over training examples, the expected reward in reinforcement learning, and the expected output of a probabilistic model.

7.2 Variance and Standard Deviation

Variance measures how spread out a random variable's values are around its expected value. High variance means the values are scattered widely; low variance means they cluster tightly around the mean.

Variance:

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$$

Standard Deviation:

$$\sigma(X) = \sqrt{\text{Var}(X)}$$

Standard deviation is in the same units as X (unlike variance, which is squared). That is why standard deviation is more interpretable — if X measures height in centimetres, σ is also in centimetres.

```
import numpy as np
from scipy.stats import describe

# Simulate 100,000 fair die rolls
np.random.seed(42)
X = np.random.randint(1, 7, size=100000)

print(f'E[X] = {X.mean():.4f} (Theory: 3.5000)')
print(f'Var(X) = {X.var():.4f} (Theory: {35/12:.4f})')
print(f'Std(X) = {X.std():.4f} (Theory: {(35/12)**0.5:.4f})')

# For a continuous variable: Normal distribution
Y = np.random.normal(loc=5, scale=2, size=100000) # mean=5, std=2
print(f'\nNormal: E[Y] = {Y.mean():.4f}')
print(f'Normal: Std(Y) = {Y.std():.4f}')
```

□ Variance in ML — The Bias-Variance Tradeoff

High variance model = overfitting. The model's predictions vary wildly on new data.

Low variance but high bias = underfitting. The model is too rigid.

The central challenge of ML is finding the right balance between bias and variance.

Regularisation techniques (L1, L2) directly reduce variance of model parameters.

Dropout in neural networks is a variance-reduction technique.

□ Mini Exercises

1. A random variable X takes values $\{0, 1, 2\}$ with probabilities $\{0.2, 0.5, 0.3\}$. Calculate $E[X]$.
2. For the same X , calculate $\text{Var}(X)$.
3. A model predicts probabilities between 0 and 1. If the predictions follow a $\text{Uniform}(0,1)$ distribution, what is $E[\text{prediction}]$ and $\text{Var}(\text{prediction})$?

✓ Answers:

1. $E[X] = 0 \times 0.2 + 1 \times 0.5 + 2 \times 0.3 = 0 + 0.5 + 0.6 = 1.1$
2. $E[X^2] = 0^2 \times 0.2 + 1^2 \times 0.5 + 2^2 \times 0.3 = 0 + 0.5 + 1.2 = 1.7$. $\text{Var}(X) = 1.7 - 1.1^2 = 1.7 - 1.21 = 0.49$
3. For $\text{Uniform}(0,1)$: $E[X] = 0.5$, $\text{Var}(X) = 1/12 \approx 0.0833$

Chapter 8: Discrete Probability Distributions

8.1 Bernoulli Distribution

The Bernoulli distribution is the simplest probability distribution. It models a single experiment that has exactly two outcomes: success (1) or failure (0). A coin flip is the canonical example.

Bernoulli Distribution:

$$P(X = 1) = p \quad \text{and} \quad P(X = 0) = 1 - p = q \quad E[X] = p \quad \text{Var}(X) = p(1-p)$$

The parameter $p \in [0, 1]$ is the probability of success. In ML, the Bernoulli distribution models binary outcomes: spam or not spam, click or no click, fraud or legitimate.

8.2 Binomial Distribution

The Binomial distribution extends Bernoulli to n independent trials. It counts the number of successes in those n trials. Each trial is a Bernoulli(p) experiment.

Binomial Distribution:

$$P(X = k) = C(n, k) \times p^k \times (1-p)^{(n-k)} \quad \text{where } C(n, k) = \frac{n!}{[k! (n-k)!]} \quad E[X] = np \quad \text{Var}(X) = np(1-p)$$

n = number of trials, k = number of successes, p = probability of success in each trial.

Example: Email Campaign

You send 100 emails. Each has a 20% chance of being opened. What is the probability that exactly 25 are opened?

$X \sim \text{Binomial}(n=100, p=0.20)$. $P(X=25) = C(100, 25) \times 0.20^{25} \times 0.80^{75} \approx 0.0505$

```
from scipy.stats import binom
import matplotlib.pyplot as plt
import numpy as np

n, p = 100, 0.20

# P(X = 25)
print(f'P(X=25) = {binom.pmf(25, n, p):.4f}')
```

P(X <= 25) – cumulative


```
print(f'P(X<=25) = {binom.cdf(25, n, p):.4f}')
```

```
# Plot the PMF
x = np.arange(0, 51)
plt.figure(figsize=(9, 4))
plt.bar(x, binom.pmf(x, n, p), color='steelblue', edgecolor='white',
alpha=0.85)
plt.axvline(n*p, color='red', linestyle='--', label=f'E[X]={n*p}')
plt.xlabel('Number of Successes (k)')
plt.ylabel('P(X = k)')
plt.title(f'Binomial Distribution n={n}, p={p}')
plt.legend() ; plt.tight_layout() ; plt.show()
```

8.3 Poisson Distribution

The Poisson distribution models the number of events that occur in a fixed interval of time or space, when events happen at a constant average rate and independently of each other.

Poisson Distribution:
$$P(X = k) = (\lambda^k \times e^{(-\lambda)}) / k! \quad E[X] = \lambda \quad \text{Var}(X) = \lambda$$

λ (lambda) is the average number of events per interval. A key property: mean equals variance for Poisson — if a count variable has mean $\neq$ variance, it's not Poisson.

Real-World Poisson Examples	λ (average rate)
Number of customer support tickets per hour	$\lambda = 5$ tickets/hour
Number of typos per 1000 words	$\lambda = 2$ typos/1000 words
Number of website visitors per minute	$\lambda = 100$ visitors/minute
Number of errors in a software deployment	$\lambda = 0.5$ errors/deployment
Number of words in a tweet	$\lambda \approx 15$ words/tweet

☐ **Poisson in ML & NLP**

Text modelling: The Poisson distribution is used to model word counts in documents.
Click prediction: Number of times a user clicks an ad follows Poisson.
Anomaly detection: If events come faster than expected, $P(X > \text{threshold})$ signals anomaly.
Poisson regression: An ML model where the output is a Poisson-distributed count.

Chapter 9: Continuous Probability Distributions

9.1 Uniform Distribution

The continuous uniform distribution assigns equal probability to all values in a range $[a, b]$. It is the most intuitive distribution — all outcomes are equally likely.

Uniform Distribution:

$$f(x) = 1/(b-a) \quad \text{for } a \leq x \leq b, \quad \text{else } f(x) = 0 \quad E[X] = (a+b)/2 \\ \text{Var}(X) = (b-a)^2/12$$

In deep learning, weights are often initialised using a uniform distribution (e.g., Glorot/Xavier initialisation). The random number generators we use in programming produce uniform random numbers.

9.2 Normal (Gaussian) Distribution — The Most Important Distribution in ML

The Normal distribution is the most important probability distribution in machine learning, statistics, and data science. It appears naturally in almost all real data due to the Central Limit Theorem (covered in Chapter 19). It has a beautiful bell-shaped curve that is symmetric around its mean.

Normal Distribution PDF:

$$f(x) = (1 / (\sigma \sqrt{2\pi})) \times \exp(- (x - \mu)^2 / (2\sigma^2)) \quad E[X] = \mu \\ \text{Var}(X) = \sigma^2$$

Two parameters fully describe a Normal distribution: μ (mu) is the mean (centre of the bell) and σ (sigma) is the standard deviation (spread of the bell). We write $X \sim N(\mu, \sigma^2)$.

The Standard Normal Distribution $N(0, 1)$

The special case where $\mu = 0$ and $\sigma = 1$ is called the standard normal. Any normal random variable $X \sim N(\mu, \sigma^2)$ can be standardised:

Standardisation (Z-score):

$$Z = (X - \mu) / \sigma \quad \rightarrow \quad Z \sim N(0, 1)$$

Z-scores tell you how many standard deviations an observation is from the mean. This is the basis of feature normalisation in machine learning.

The 68-95-99.7 Rule

Range	Probability Contained
$\mu \pm 1\sigma$	~68.3% of data
$\mu \pm 2\sigma$	~95.4% of data
$\mu \pm 3\sigma$	~99.7% of data
Beyond $\pm 3\sigma$	Only ~0.3% (these are extreme outliers)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

mu, sigma = 170, 10 # e.g., human heights in cm
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 400)
y = norm.pdf(x, mu, sigma)

plt.figure(figsize=(10, 5))
plt.plot(x, y, 'navy', lw=2, label=f'N( $\mu$ ={mu},  $\sigma$ ={sigma})')

# Fill 68%, 95%, 99.7% regions
for (lo, hi, col, lab) in [
    (mu-sigma, mu+sigma, '#2196F3', '68%'),
    (mu-2*sigma, mu+2*sigma, '#90CAF9', '95%'),
    (mu-3*sigma, mu+3*sigma, '#E3F2FD', '99.7%')]:
    mask = (x >= lo) & (x <= hi)
    plt.fill_between(x[mask], y[mask], alpha=0.5, color=col, label=lab)

plt.xlabel('Height (cm)') ; plt.ylabel('Probability Density')
plt.title('Normal Distribution - 68-95-99.7 Rule')
plt.legend() ; plt.tight_layout() ; plt.show()

# Compute percentages
print(f'P(within 1 $\sigma$ ) = {norm.cdf(1) - norm.cdf(-1):.4f}')
print(f'P(within 2 $\sigma$ ) = {norm.cdf(2) - norm.cdf(-2):.4f}')
print(f'P(within 3 $\sigma$ ) = {norm.cdf(3) - norm.cdf(-3):.4f}')
```

Why the Normal Distribution is Central to ML

- Feature normalisation: StandardScaler centres features to $N(0,1)$ for better training.
- Weight initialisation: Glorot/He initialisation uses $N(0, \sigma^2)$ to set initial network weights.
- Gaussian Naive Bayes: Assumes features are normally distributed given the class.
- Residual analysis: Errors in linear regression should follow $N(0, \sigma^2)$.
- Central Limit Theorem: Any average of many samples approaches Normal distribution.
- Gaussian Processes: Entire ML models built on multivariate normal distributions.

Chapter 10: PMF, PDF & CDF — Describing Distributions

10.1 Probability Mass Function (PMF)

A Probability Mass Function (PMF) gives the probability that a discrete random variable takes exactly a specific value. It maps each possible value to its probability.

PMF Properties:

$p(x) = P(X = x)$ for each possible value x Conditions: $p(x) \geq 0$
for all x , and $\sum p(x) = 1$

10.2 Probability Density Function (PDF)

For continuous random variables, we cannot assign probability to exact values (since $P(X = \text{exact value}) = 0$). Instead, we use a Probability Density Function (PDF), which tells us the relative likelihood of the variable being near a value.

PDF Properties:

$f(x) \geq 0$ for all x $P(a \leq X \leq b) = \int [a \text{ to } b] f(x) dx$ $\int [-\infty \text{ to } +\infty] f(x) dx = 1$

The key insight: with a PDF, you never ask 'What is the probability that X equals exactly 1.5?' You always ask 'What is the probability that X falls between 1.4 and 1.6?' (or some interval).

10.3 Cumulative Distribution Function (CDF)

The CDF $F(x)$ gives the probability that the random variable X takes a value less than or equal to x . It accumulates probability from $-\infty$ up to x .

CDF Definition:

$F(x) = P(X \leq x)$

The CDF is always non-decreasing, starts at 0 for $x \rightarrow -\infty$, and reaches 1 for $x \rightarrow +\infty$. For any interval: $P(a < X \leq b) = F(b) - F(a)$.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm, binom
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Discrete PMF - Binomial
x_d = np.arange(0, 21)
pmf = binom.pmf(x_d, n=20, p=0.4)
axes[0,0].bar(x_d, pmf, color='steelblue', edgecolor='white')
axes[0,0].set_title('Binomial PMF (n=20, p=0.4)')
axes[0,0].set_xlabel('k') ; axes[0,0].set_ylabel('P(X=k)')

# Discrete CDF - Binomial
cdf_d = binom.cdf(x_d, n=20, p=0.4)
axes[0,1].step(x_d, cdf_d, where='post', color='steelblue', lw=2)
axes[0,1].set_title('Binomial CDF')
axes[0,1].set_xlabel('k') ; axes[0,1].set_ylabel('P(X≤k)')

# Continuous PDF - Normal
x_c = np.linspace(-4, 4, 300)
axes[1,0].plot(x_c, norm.pdf(x_c), color='crimson', lw=2)
axes[1,0].fill_between(x_c, norm.pdf(x_c), alpha=0.3, color='crimson')
axes[1,0].set_title('Normal PDF')

# Continuous CDF - Normal
axes[1,1].plot(x_c, norm.cdf(x_c), color='crimson', lw=2)
axes[1,1].set_title('Normal CDF')
axes[1,1].set_xlabel('x') ; axes[1,1].set_ylabel('P(X≤x)')

plt.tight_layout() ; plt.show()
```

□ PMF / PDF / CDF in Machine Learning

CDF → Percentile calculations: 'What is the 95th percentile of prediction scores?'

PDF → Anomaly detection: 'How likely is this data point under the learned distribution?'

CDF → Calibration: 'Are the model's predicted probabilities actually reliable?'

PMF → Language models: 'What is the probability of this next word?'

Chapter 11: Joint, Marginal & Conditional Distributions

11.1 Joint Probability Distribution

When we have two or more random variables, we need to describe how they behave together. The joint distribution $P(X, Y)$ gives the probability of each combination of values for X and Y simultaneously.

Joint PMF (discrete):

$$P(X = x, Y = y) \quad \text{for all combinations of } x \text{ and } y$$

Joint PDF (continuous):

$$f(x, y) \quad \text{where} \quad P(a \leq X \leq b, c \leq Y \leq d) = \iint f(x, y) \, dx \, dy$$

Example: Weather & Traffic

	Traffic: Light	Traffic: Heavy
Weather: Sunny	$P = 0.45$	$P = 0.15$
Weather: Rainy	$P = 0.05$	$P = 0.35$

Total: $0.45 + 0.15 + 0.05 + 0.35 = 1.00$ ✓

11.2 Marginal Distribution

The marginal distribution of X is obtained by summing (or integrating) the joint distribution over all values of Y . It 'marginalises out' the other variable.

Marginal Distribution:

$$P(X = x) = \sum_y P(X = x, Y = y) \quad (\text{discrete}) \quad f(x) = \int f(x, y) \, dy$$

(continuous)

From the weather-traffic table: $P(\text{Sunny}) = 0.45 + 0.15 = 0.60$. $P(\text{Heavy traffic}) = 0.15 + 0.35 = 0.50$.

11.3 Independence of Random Variables

Two random variables X and Y are independent if and only if their joint distribution factors into the product of their marginal distributions:

Independence of Random Variables:

X and Y are independent $\Leftrightarrow P(X, Y) = P(X) \times P(Y)$

In the weather-traffic example: $P(\text{Sunny}) \times P(\text{Light}) = 0.60 \times 0.50 = 0.30$, but $P(\text{Sunny}, \text{Light}) = 0.45$. These don't match, so weather and traffic are NOT independent — rain causes heavier traffic.

□ **Joint Distributions in Deep Learning**

Variational Autoencoders (VAEs) learn the joint distribution $P(X, Z)$ of data X and latent variable Z .

Generative Adversarial Networks (GANs) approximate the joint distribution of real data.

Multivariate Gaussian models use joint distributions over multiple features simultaneously.

Conditional Independence: In Bayesian networks, joint distributions factorise along DAG edges.

Chapter 12: Likelihood & Maximum Likelihood Estimation

12.1 Probability vs. Likelihood

This is one of the most important but often confusing distinctions in statistics and machine learning. Both probability and likelihood involve the same formula, but they answer fundamentally different questions.

Probability $P(\text{data} \mid \text{parameters})$	Likelihood $L(\text{parameters} \mid \text{data})$
Parameters θ are fixed and KNOWN	Data X is fixed and OBSERVED
Data X is the variable	Parameters θ are the variable
'Given these model parameters, how likely is this data?'	'Given this observed data, how well do these parameters explain it?'
Ranges from 0 to 1, sums to 1	Can be any positive number, does NOT sum to 1
Used for prediction	Used for model fitting / training

Likelihood Definition:
 $L(\theta \mid X) = P(X \mid \theta)$ (Same formula, different perspective – parameters vary, data is fixed)

□ **The Likelihood Analogy**

Imagine you found footprints in the snow. You're not asking 'If a bear was here, how likely are these prints?'

You're asking: 'A bear ($\theta=\text{bear}$), a dog ($\theta=\text{dog}$), or a human ($\theta=\text{human}$) — which BEST explains these footprints (data)?'

The parameters that make the observed data most likely are the maximum likelihood estimates. This is exactly what ML training does: find model parameters that maximise likelihood of the training data.

12.2 Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE) is the most widely used method for fitting probability models to data. The idea is to find the parameter values θ^* that make the observed data as probable as possible.

MLE Objective:
 $\theta^* = \operatorname{argmax}_{\theta} L(\theta \mid X) = \operatorname{argmax}_{\theta} P(X \mid \theta)$

In practice, we maximise the log-likelihood instead. This is mathematically equivalent (log is monotone) but much easier to compute — it turns products into sums.

Log-Likelihood (used in practice):

$\ell(\theta \mid X) = \log L(\theta \mid X) = \sum \log P(x_i \mid \theta)$ (Sum over all training examples x_i)

MLE for a Coin: Step-by-Step

You flip a coin 100 times and observe 63 heads. What is the MLE estimate of the probability p of heads?

21. Likelihood: $L(p) = p^{63} \times (1-p)^{37}$ (Binomial likelihood, ignoring constant)

22. Log-likelihood: $\ell(p) = 63 \cdot \log(p) + 37 \cdot \log(1-p)$

23. Differentiate and set to zero: $d\ell/dp = 63/p - 37/(1-p) = 0$

24. Solve: $p^* = 63/100 = 0.63$

The MLE estimate is simply the observed proportion. This confirms intuition: the best estimate for p is the proportion of successes in your data.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar

heads, tails = 63, 37

# Log-likelihood function (negative, for minimisation)
def neg_log_likelihood(p):
    if p <= 0 or p >= 1: return 1e10
    return -(heads * np.log(p) + tails * np.log(1-p))

result = minimize_scalar(neg_log_likelihood, bounds=(0.01, 0.99),
method='bounded')
print(f'MLE estimate: p* = {result.x:.4f}') # Should be 0.63

# Plot log-likelihood curve
p_vals = np.linspace(0.01, 0.99, 300)
ll_vals = [heads*np.log(p) + tails*np.log(1-p) for p in p_vals]
plt.figure(figsize=(8, 4))
plt.plot(p_vals, ll_vals, color='steelblue', lw=2)
plt.axvline(result.x, color='red', linestyle='--', label=f'MLE
p*={result.x:.2f}')
plt.xlabel('p') ; plt.ylabel('Log-Likelihood')
plt.title('Log-Likelihood for Coin Flip (63 heads, 37 tails)')
plt.legend() ; plt.tight_layout() ; plt.show()
```

12.3 MLE and Neural Network Training

This connection is crucial: training a neural network by minimising cross-entropy loss is EXACTLY the same as maximising the likelihood of the training data.

Cross-Entropy Loss (Binary Classification):
$$L = -\sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)] = -\log\text{-likelihood under Bernoulli model}$$

This is why we use cross-entropy loss in classification problems — it has a deep probabilistic justification through MLE.

Chapter 13: Entropy & Information Theory

13.1 What is Entropy?

Entropy is a measure of uncertainty or randomness in a probability distribution. It quantifies how much 'surprise' or 'information' you receive when you observe an outcome. High entropy means high uncertainty; low entropy means predictability.

□ The Surprise Analogy

If someone tells you 'the sun rose this morning' — not surprising. Low information.

If someone tells you 'it snowed in Karachi in July' — very surprising. High information.

Information content of event $A = \log(1/P(A)) = -\log(P(A))$

Events with low probability carry more information (they are more surprising when they occur).

Entropy is the AVERAGE information content over all possible events.

Shannon Entropy:

$H(X) = -\sum P(x) \cdot \log_2 P(x)$ for all x (Use natural log for 'nats', $\log_2$ for 'bits')

Entropy Examples

Distribution	Entropy (bits)	Interpretation
Fair coin: $P(H)=P(T)=0.5$	$H = 1.0$ bit	Maximum uncertainty — truly unpredictable
Biased coin: $P(H)=0.9, P(T)=0.1$	$H \approx 0.47$ bits	Low uncertainty — usually lands heads
Certain outcome: $P(H)=1.0, P(T)=0.0$	$H = 0.0$ bits	Zero uncertainty — deterministic
Fair 6-sided die	$H \approx 2.58$ bits	More outcomes → more uncertainty

13.2 Cross-Entropy

Cross-entropy measures how well a predicted distribution Q approximates a true distribution P . It is the basis of the loss function for classification in neural networks.

Cross-Entropy:

$H(P, Q) = -\sum P(x) \cdot \log Q(x)$

When $Q = P$ (perfect prediction), cross-entropy equals entropy $H(P)$. When Q is wrong, cross-entropy is higher. The neural network training process minimises cross-entropy — pushing Q toward P .

13.3 KL Divergence

The Kullback-Leibler (KL) divergence measures how much one probability distribution differs from another. It is the difference between cross-entropy and entropy.

KL Divergence:

$$D_{\text{KL}}(P \parallel Q) = \sum P(x) \cdot \log(P(x) / Q(x)) = H(P, Q) - H(P)$$

KL divergence is always ≥ 0 , and equals 0 only when $P = Q$. It is used in Variational Autoencoders as a regularisation term, and in many other generative models.

```
import numpy as np
from scipy.stats import entropy as scipy_entropy

# Entropy examples
fair_coin = np.array([0.5, 0.5])
biased    = np.array([0.9, 0.1])
certain   = np.array([1.0 - 1e-10, 1e-10]) # avoid log(0)
fair_die   = np.array([1/6]*6)

for name, dist in [('Fair coin', fair_coin), ('Biased coin', biased),
                  ('Certain', certain), ('Fair die', fair_die)]:
    h = scipy_entropy(dist, base=2)
    print(f'{name:15s} H = {h:.4f} bits')

# Cross-entropy and KL divergence
P = np.array([0.8, 0.2]) # True labels
Q = np.array([0.6, 0.4]) # Model predictions

cross_entropy = -np.sum(P * np.log(Q))
kl_div = scipy_entropy(P, Q) # KL(P||Q)
print(f'\nCross-entropy H(P,Q) = {cross_entropy:.4f}')
print(f'KL divergence D_KL = {kl_div:.4f}')
print(f'Entropy H(P) = {scipy_entropy(P):.4f}')
```

13.4 Information Gain — Decision Trees

Information Gain measures how much a feature reduces entropy in a dataset. Decision trees use Information Gain (or the related Gini impurity) to select the best feature to split on at each node.

Information Gain:

$IG(S, A) = H(S) - \sum (|S_v|/|S|) \times H(S_v)$ where S_v are the subsets after splitting on feature A

A feature with high Information Gain splits the data into groups that are more pure (lower entropy) than before. The decision tree greedily selects the feature with the highest Information Gain.

Chapter 14: Law of Large Numbers & Central Limit Theorem

14.1 The Law of Large Numbers (LLN)

The Law of Large Numbers states that as the number of trials of a random experiment increases, the sample mean converges to the true population mean (expected value). The more data you have, the more stable and accurate your estimates become.

Law of Large Numbers:

$$\bar{X} = (1/n) \sum X_i \rightarrow E[X] = \mu \quad \text{as } n \rightarrow \infty$$

LLN in Practice

Casino: The house always wins — in the short run, anything can happen; in the long run, the casino edge dominates.

ML training: After seeing enough training examples, gradient estimates stabilise.

A/B testing: You need enough samples before concluding one version is better.

Monte Carlo: The more random samples you draw, the closer your estimate gets to the true value.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
true_mean = 3.5 # Expected value of fair die
N = 5000
rolls = np.random.randint(1, 7, N)

# Cumulative mean after each roll
cumulative_mean = np.cumsum(rolls) / np.arange(1, N+1)

plt.figure(figsize=(10, 4))
plt.plot(cumulative_mean, color='steelblue', lw=1.5, label='Sample mean')
plt.axhline(true_mean, color='red', linestyle='--', lw=2, label=f'True E[X]={true_mean}')
plt.xlabel('Number of Rolls') ; plt.ylabel('Sample Mean')
plt.title('Law of Large Numbers – Dice Roll Convergence')
plt.legend() ; plt.tight_layout() ; plt.show()
```

14.2 Central Limit Theorem (CLT)

The Central Limit Theorem is arguably the most important theorem in all of statistics. It states that the sum (or mean) of a large number of independent random variables — regardless of their original distribution — converges to a Normal distribution.

Central Limit Theorem:

If $X_1, X_2, \dots, X_n$ are i.i.d. with mean μ and variance σ^2 , then as $n \rightarrow \infty$: $\sqrt{n} (X - \mu) / \sigma \sim N(0, 1)$

In other words: take ANY distribution. Sample n observations from it. Compute their average. Repeat this many times. The distribution of those averages approaches a Normal distribution, no matter what the original distribution was.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)
n_experiments = 10000

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
titles = ['n=1 (raw)', 'n=5 (average of 5)', 'n=30 (average of 30)']

for i, n in enumerate([1, 5, 30]):
    # Draw from exponential (skewed, non-normal)
    samples = np.random.exponential(scale=2, size=(n_experiments, n))
    means = samples.mean(axis=1)

    axes[i].hist(means, bins=60, density=True, color='steelblue',
                 edgecolor='white', alpha=0.75)
    # Overlay normal fit
    mu_fit, std_fit = means.mean(), means.std()
    x = np.linspace(means.min(), means.max(), 200)
    axes[i].plot(x, norm.pdf(x, mu_fit, std_fit), 'r-', lw=2)
    axes[i].set_title(titles[i])

plt.suptitle('CLT: Averages of Exponential Distribution Become Normal')
plt.tight_layout() ; plt.show()
```

□ CLT in Machine Learning

Mini-batch gradient descent: Averaging gradients over a batch approximates the true gradient via CLT.

Confidence intervals: Model evaluation metrics (accuracy, AUC) have confidence intervals based on CLT.

A/B testing: CLT justifies using z-tests and t-tests for comparing model performance.

Ensemble methods: Averaging many weak classifiers reduces variance — a consequence of CLT.

Chapter 15: Naive Bayes Classifier

15.1 What is Naive Bayes?

Naive Bayes is a probabilistic machine learning classifier based directly on Bayes' Theorem. It assumes that all features are conditionally independent given the class label — this is the 'naive' assumption. Despite this strong simplification, Naive Bayes works remarkably well in practice, especially for text classification.

Naive Bayes Classification Rule:

$$P(C \mid x_1, x_2, \dots, x_n) \propto P(C) \times \prod P(x_i \mid C) \text{ (product over all features } x_i)$$

The classifier picks the class C that maximises this expression — the class with the highest posterior probability given all the features.

15.2 Step-by-Step: Spam Detection

Training data: 1000 emails — 600 ham, 400 spam. From these emails:

Word	P(word ham)	P(word spam)
'meeting'	0.15	0.02
'free'	0.01	0.30
'money'	0.005	0.25
'report'	0.12	0.01

New email contains: 'free money'. Classify as spam or ham?

$$25. P(\text{ham}) = 600/1000 = 0.6, \quad P(\text{spam}) = 400/1000 = 0.4$$

$$26. P(\text{ham} \mid \text{free, money}) \propto P(\text{ham}) \times P(\text{free} \mid \text{ham}) \times P(\text{money} \mid \text{ham}) = 0.6 \times 0.01 \times 0.005 = 0.000030$$

$$27. P(\text{spam} \mid \text{free, money}) \propto P(\text{spam}) \times P(\text{free} \mid \text{spam}) \times P(\text{money} \mid \text{spam}) = 0.4 \times 0.30 \times 0.25 = 0.030000$$

$$28. \text{Normalise: } P(\text{spam} \mid \dots) = 0.030 / (0.030 + 0.000030) = 0.9990$$

→ Decision: SPAM with 99.9% confidence. ✓

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Gaussian Naive Bayes on Iris dataset
```

```
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

print(classification_report(y_test, y_pred,
                           target_names=iris.target_names))
print(f'Accuracy: {gnb.score(X_test, y_test):.4f}')

# Predicted probabilities (not just class labels)
probs = gnb.predict_proba(X_test[:5])
for i, p in enumerate(probs):
    print(f'Sample {i}: {dict(zip(iris.target_names, p.round(3)))}')
```

Chapter 16: Gaussian (Normal) Models in Machine Learning

16.1 Gaussian Naive Bayes

When features are continuous, Naive Bayes assumes each feature follows a Normal distribution given the class. For each class C and feature x_i , we estimate μ_{iC} (mean) and σ_{iC}^2 (variance) from training data.

Gaussian Naive Bayes Likelihood:

$$P(x_i | C) = (1/\sqrt{2\pi\sigma_{iC}^2}) \times \exp(-(x_i - \mu_{iC})^2 / (2\sigma_{iC}^2))$$

16.2 Gaussian Mixture Models (GMM)

A Gaussian Mixture Model represents a probability distribution as a weighted sum of multiple Gaussian distributions. It is a powerful unsupervised learning algorithm for clustering when clusters are not spherical or have different sizes.

GMM Density:

$$f(x) = \sum w_k \times N(x; \mu_k, \Sigma_k) \quad \text{where } \sum w_k = 1$$

Each component k has weight w_k , mean μ_k , and covariance Σ_k . GMMs are trained using the Expectation-Maximisation (EM) algorithm — a direct application of probability theory.

```
from sklearn.mixture import GaussianMixture
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Generate data from two overlapping Gaussians
group1 = np.random.multivariate_normal([2, 2], [[1, 0.5], [0.5, 1]], 300)
group2 = np.random.multivariate_normal([6, 4], [[1.5, -0.3], [-0.3, 1]], 300)
X = np.vstack([group1, group2])

# Fit GMM
gmm = GaussianMixture(n_components=2, random_state=42)
gmm.fit(X)
labels = gmm.predict(X)

plt.figure(figsize=(8, 5))
```

```
plt.scatter(X[labels==0, 0], X[labels==0, 1], alpha=0.5, label='Component 1',
s=20)
plt.scatter(X[labels==1, 0], X[labels==1, 1], alpha=0.5, label='Component 2',
s=20)
plt.scatter(gmm.means_[:, 0], gmm.means_[:, 1],
            c='red', s=200, marker='*', label='GMM Means')
plt.title('Gaussian Mixture Model Clustering')
plt.legend() ; plt.tight_layout() ; plt.show()

print('GMM Means:') ; print(gmm.means_.round(2))
```

16.3 Normal Distribution in Neural Network Weight Initialisation

Before training begins, neural network weights need to be initialised. Using $N(0, \sigma^2)$ is standard practice. Two well-known initialisation schemes are:

Initialisation Scheme	Distribution Used	Best For
Xavier / Glorot	$N(0, 1/n_i)$ or Uniform($-\sqrt{6/(n_i+n_{ou})}$, ...)	Sigmoid, Tanh activations
He Initialisation	$N(0, 2/n_i)$	ReLU activations
Lecun Initialisation	$N(0, 1/n_i)$	SELU activations

□ Why Weight Initialisation Matters

Too large: Activations explode — gradients vanish or explode, training fails.

Too small: Activations shrink — gradients vanish, training fails.

Just right (via Gaussian): Signal propagates through layers with stable variance.

The correct variance depends on the number of neurons — probability theory tells us why.

Chapter 17: Probabilistic Thinking in AI

17.1 Uncertainty in AI Systems

Real AI systems must handle uncertainty gracefully. A model that outputs a confident but wrong prediction is worse than a model that says 'I'm not sure.' Modern AI distinguishes between two types of uncertainty:

Type of Uncertainty	Description	Example
Aleatoric Uncertainty	Randomness inherent in the data — cannot be reduced by more data	Sensor noise, human variability, natural randomness
Epistemic Uncertainty	Uncertainty due to lack of knowledge — CAN be reduced with more data or better model	Model not trained on enough examples, unknown territory

17.2 Probabilistic ML Models vs. Deterministic

A deterministic model outputs a single prediction: 'This is a cat.' A probabilistic model outputs a distribution: 'This is a cat with 87% probability, a dog with 10% probability, and something else with 3% probability.' Probabilistic outputs are far more useful in practice.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate binary classification data
X, y = make_classification(n_samples=1000, n_features=10,
                          random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Logistic Regression - a probabilistic classifier
lr = LogisticRegression(random_state=42)
lr.fit(X_train, y_train)

# Predict probabilities (not just labels)
probs = lr.predict_proba(X_test[:10])
print('Sample   P(class=0)   P(class=1)')
for i, (p0, p1) in enumerate(probs):
    print(f'   {i:2d}         {p0:.4f}         {p1:.4f}')

# Calibration check
from sklearn.calibration import calibration_curve
# ... (calibration plot code)
```

17.3 Softmax — Converting Scores to Probabilities

In neural networks, the final layer for multi-class classification uses the Softmax function, which converts raw output scores (logits) into a valid probability distribution that sums to 1.

Softmax Function:

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j) \quad \text{for each class } i$$

The exponential function ensures all values are positive, and dividing by the sum ensures they sum to 1. The class with the highest softmax output is the predicted class.

```
import numpy as np

def softmax(z):
    """Numerically stable softmax implementation."""
    z_shifted = z - np.max(z) # subtract max for numerical stability
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum()

# Example: 3-class classification raw scores
logits = np.array([2.1, 0.5, -1.3]) # raw neural net outputs
probs = softmax(logits)

print('Raw logits:      ', logits)
print('Softmax probs:   ', probs.round(4))
print('Sum of probs:    ', probs.sum().round(6)) # Should be 1.0
print('Predicted class:', np.argmax(probs))
```

17.4 Bayesian Deep Learning

Traditional neural networks give point estimates of parameters (single weight values). Bayesian neural networks instead maintain a probability distribution over weights, naturally quantifying uncertainty.

□ Bayesian Neural Networks

Instead of one set of weights θ , a Bayesian NN learns $P(\theta \mid \text{data})$.

At prediction time, you sample many weight configurations and average their outputs.

The variance of those outputs tells you the model's uncertainty.

Useful for medical AI, autonomous vehicles — any domain where uncertainty matters.

Implemented via techniques like Monte Carlo Dropout, Variational Inference.

Chapter 18: Probability with Python — Practical Applications

18.1 Simulating Basic Probability Experiments

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

np.random.seed(42)

# ——— Coin Flips ———
n_flips = 10000
coins = np.random.choice(['H', 'T'], size=n_flips, p=[0.5, 0.5])
p_heads = (coins == 'H').mean()
print(f'Coin: P(Heads) simulated = {p_heads:.4f} (theory = 0.5000)')

# ——— Dice Rolls ———
dice = np.random.randint(1, 7, n_flips)
for face in range(1, 7):
    print(f' P(dice={face}) = {(dice==face).mean():.4f}')

# ——— Birthday Problem via Monte Carlo ———
def birthday_collision_prob(n_people, n_sim=10000):
    count = 0
    for _ in range(n_sim):
        birthdays = np.random.randint(1, 366, n_people)
        if len(set(birthdays)) < n_people: # collision!
            count += 1
    return count / n_sim

for n in [10, 23, 30, 50, 70]:
    p = birthday_collision_prob(n, n_sim=5000)
    print(f'Birthday collision, n={n:3d}: P = {p:.3f}')
```

18.2 Working with Distributions

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ——— Comparing Multiple Distributions ———
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
```

```
distributions = [
    ('Bernoulli(p=0.7)', stats.bernoulli(p=0.7), 'discrete'),
    ('Binomial(20,0.4)', stats.binom(n=20, p=0.4), 'discrete'),
    ('Poisson( $\lambda$ =5)', stats.poisson(mu=5), 'discrete'),
    ('Uniform(0,1)', stats.uniform(0, 1), 'continuous'),
    ('Normal(0,1)', stats.norm(0, 1), 'continuous'),
    ('Exponential(1)', stats.expon(scale=1), 'continuous'),
]

for ax, (name, dist, dtype) in zip(axes.flat, distributions):
    if dtype == 'discrete':
        x = np.arange(0, 30)
        ax.bar(x, dist.pmf(x), color='steelblue', edgecolor='white',
               alpha=0.8)
    else:
        x = np.linspace(dist.ppf(0.001), dist.ppf(0.999), 300)
        ax.plot(x, dist.pdf(x), lw=2, color='crimson')
        ax.fill_between(x, dist.pdf(x), alpha=0.3, color='crimson')
    ax.set_title(name)
    ax.set_xlabel('x') ; ax.set_ylabel('Probability')

plt.suptitle('Common Probability Distributions in ML', fontsize=14)
plt.tight_layout() ; plt.show()
```

18.3 Visualising Bayes' Theorem Updates

```
import numpy as np
import matplotlib.pyplot as plt

# Prior: uniform belief that P(heads) could be any value 0..1
# Data: we observe coin flips. Update belief using Bayes!

p_vals = np.linspace(0.01, 0.99, 500)

def posterior(p_vals, heads, tails):
    likelihood = p_vals**heads * (1 - p_vals)**tails
    posterior = likelihood # × uniform prior (constant)
    return posterior / posterior.sum() # normalise

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
scenarios = [(0, 0, 'Prior (no data)'),
              (3, 7, 'After 10 flips: 3H, 7T'),
              (60, 40, 'After 100 flips: 60H, 40T')]

for ax, (h, t, title) in zip(axes, scenarios):
    if h == 0 and t == 0:
        post = np.ones_like(p_vals) / len(p_vals)
```



```
    else:
        post = posterior(p_vals, h, t)
        ax.plot(p_vals, post, lw=2, color='steelblue')
        ax.fill_between(p_vals, post, alpha=0.3, color='steelblue')
    if h + t > 0:
        ax.axvline(h/(h+t), color='red', linestyle='--',
                   label=f'MLE = {h/(h+t):.2f}')
        ax.legend()
    ax.set_title(title) ; ax.set_xlabel('p (P(heads))')
    ax.set_ylabel('Posterior density')

plt.suptitle('Bayesian Updating: Belief About Coin Fairness')
plt.tight_layout() ; plt.show()
```

Chapter 19: Mini Project — Build a Naive Bayes Text Classifier

19.1 Project Overview

In this mini project, you will build a complete Naive Bayes classifier from scratch and also using scikit-learn. You will classify SMS messages as spam or ham (legitimate) based on their word content. This project brings together conditional probability, Bayes' Theorem, MLE, and practical Python coding.

19.2 Step-by-Step Implementation

```
# =====  
# MINI PROJECT: Naive Bayes SMS Spam Classifier  
# =====  
  
import numpy as np  
from collections import defaultdict  
import math  
  
# — 1. Training Data —  
training_data = [  
    ('ham', 'call me when you get home'),  
    ('ham', 'dinner tonight at 7pm meeting'),  
    ('ham', 'are you free for a quick meeting'),  
    ('ham', 'the report is ready for review'),  
    ('ham', 'can we reschedule our meeting'),  
    ('spam', 'free money click here now win'),  
    ('spam', 'win free cash money now click'),  
    ('spam', 'you won a free prize claim now'),  
    ('spam', 'make money fast free offer click'),  
    ('spam', 'free credit offer win money now'),  
]  
  
# — 2. Build Vocabulary & Count Words —  
word_count = {'ham': defaultdict(int), 'spam': defaultdict(int)}  
class_count = {'ham': 0, 'spam': 0}  
vocab = set()  
  
for label, text in training_data:  
    class_count[label] += 1  
    for word in text.split():  
        word_count[label][word] += 1  
        vocab.add(word)  
  
total = sum(class_count.values())  
V = len(vocab) # vocabulary size (for Laplace smoothing)
```

```
# — 3. Naive Bayes Classifier with Laplace Smoothing —
def classify(text):
    words = text.split()
    best_label, best_score = None, float('-inf')

    for label in ['ham', 'spam']:
        # Log prior probability
        log_prob = math.log(class_count[label] / total)

        # Log likelihood of each word (Laplace +1 smoothing)
        total_words = sum(word_count[label].values())
        for word in words:
            count = word_count[label].get(word, 0) + 1
            log_prob += math.log(count / (total_words + V))

        if log_prob > best_score:
            best_score, best_label = log_prob, label

    return best_label

# — 4. Test on New Messages —
test_messages = [
    'win free money now',
    'can we schedule a meeting',
    'free offer click here',
    'when are you free tonight dinner',
]

print('=' * 45)
print('Naive Bayes SMS Classifier Results')
print('=' * 45)
for msg in test_messages:
    label = classify(msg)
    print(f'{label.upper():5s} | {msg}')
```

19.3 Using Scikit-learn Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import cross_val_score
import numpy as np

# Training data
texts = [text for _, text in training_data]
labels = [lbl for lbl, _ in training_data]

# Build a proper NLP pipeline
```

```
pipeline = Pipeline([
    ('vect', CountVectorizer()),          # bag-of-words
    ('tfidf', TfidfTransformer()),       # tf-idf weighting
    ('clf', MultinomialNB(alpha=1.0)),    # Laplace smoothing
])

pipeline.fit(texts, labels)

# Predict on new messages
new_msgs = ['win cash free prize', 'meeting at 3pm tomorrow']
preds     = pipeline.predict(new_msgs)
probs     = pipeline.predict_proba(new_msgs)

for msg, pred, prob in zip(new_msgs, preds, probs):
    print(f'{pred.upper():5s} (P={max(prob):.3f}) | {msg}')
```

Chapter 20: Summary & Probability Cheat Sheet

20.1 Complete Formula Reference

Concept	Formula	Parameters	ML Use
Basic Probability	$P(A) = \text{favourable} / \text{total}$	Equally likely outcomes	Class balance
Complement	$P(A') = 1 - P(A)$	—	One-vs-rest
Addition	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$	Events A, B	Combined events
Multiplication (indep.)	$P(A \cap B) = P(A) \cdot P(B)$	Independent events	Feature independence
Conditional	$P(A B) = P(A \cap B) / P(B)$	B has occurred	Classifier outputs
Bayes' Theorem	$P(A B) = P(B A)P(A)/P(B)$	Prior $P(A)$	Bayesian inference
Expected Value	$E[X] = \sum x \cdot P(x)$	Discrete dist.	Loss, reward
Variance	$\text{Var}(X) = E[X^2] - (E[X])^2$	—	Overfitting
Bernoulli	$P(X=1)=p, P(X=0)=1-p$	$p \in [0,1]$	Binary outcomes
Binomial	$C(n,k)p^k(1-p)^{(n-k)}$	n trials, p	Count successes
Poisson	$e^{(-\lambda)}\lambda^k/k!$	$\lambda = \text{avg rate}$	Count events
Normal PDF	$(1/\sigma\sqrt{2\pi})\exp(-(x-\mu)^2/2\sigma^2)$	μ, σ^2	Feature dist.
Entropy	$H(X) = -\sum P(x)\log P(x)$	—	Decision trees
Cross-Entropy	$H(P,Q) = -\sum P(x)\log Q(x)$	P true, Q pred	Neural net loss
MLE	$\theta^* = \text{argmax} \sum \log P(x_i \theta)$	Model θ	Training
Softmax	$\exp(z_i) / \sum \exp(z_j)$	Logits z	Multiclass output

20.2 Distributions Quick Reference

Distribution	Use When	$E[X]$	$\text{Var}(X)$
Bernoulli(p)	Single binary trial	p	$p(1-p)$
Binomial(n,p)	n binary trials	np	$np(1-p)$
Poisson(λ)	Counting rare events	λ	λ
Uniform(a,b)	Equal likelihood	$(a+b)/2$	$(b-a)^2/12$
Normal(μ, σ^2)	Continuous, symmetric	μ	σ^2
Exponential(λ)	Time between events	$1/\lambda$	$1/\lambda^2$

20.3 Learning Roadmap — What to Study Next

Congratulations on completing this guide! Here is a suggested roadmap to continue your journey in probabilistic machine learning:

- 29. Statistics for ML — hypothesis testing, confidence intervals, sampling
- 30. Linear Algebra for ML — vectors, matrices, eigenvalues
- 31. Calculus for ML — gradients, optimisation, backpropagation
- 32. Machine Learning Algorithms — build on your probabilistic foundation
- 33. Deep Learning — neural networks, cross-entropy loss, softmax
- 34. Bayesian Machine Learning — Gaussian Processes, Variational Inference
- 35. Probabilistic Graphical Models — Bayesian networks, hidden Markov models

□ Key Takeaways

Probability is the mathematical language of uncertainty — the foundation of all ML.
Bayes' Theorem allows us to update beliefs with evidence — the basis of Bayesian ML.
The Normal distribution appears everywhere due to the Central Limit Theorem.
MLE is the theoretical justification for why we minimise cross-entropy in neural networks.
Entropy measures uncertainty — directly used in decision trees and information theory.
Every ML model is a probability machine: it outputs probabilities, not certainties.

Appendix: Key Python Libraries for Probability

Library / Function	Use in Probability & ML
<code>numpy.random</code>	Random experiments, simulations, sampling
<code>scipy.stats</code>	Distributions (pmf, pdf, cdf), statistical tests
<code>matplotlib.pyplot</code>	Visualising distributions, histograms, density plots
<code>seaborn</code>	Beautiful statistical visualisations with less code
<code>sklearn.naive_bayes</code>	GaussianNB, MultinomialNB, BernoulliNB classifiers
<code>sklearn.calibration</code>	Calibration curves — are predicted probabilities reliable?
<code>sklearn.mixture</code>	Gaussian Mixture Models (GMM)
<code>sklearn.linear_model</code>	LogisticRegression — probabilistic linear classifier
<code>scipy.optimize</code>	MLE parameter estimation via optimisation
<code>scipy.special</code>	Log-sum-exp, betaln — numerical stability helpers

□ Recommended Resources

Book: 'Pattern Recognition and Machine Learning' by Bishop (Bishop's Bible for probabilistic ML)

Book: 'The Elements of Statistical Learning' by Hastie, Tibshirani, Friedman (free online)

Book: 'Probability Theory: The Logic of Science' by E.T. Jaynes (deep Bayesian perspective)

Course: Khan Academy — Probability and Statistics (beginner friendly)

Course: MIT OpenCourseWare 6.041 — Probabilistic Systems Analysis

Course: fast.ai Practical Deep Learning — applies probability throughout